

M serving, multi-instance load balancing, vLLM, hybrid routing, prefix cache affinity, admission control, NLMS, open-source systems

DIO: Hybrid Cost Routing, Session Affinity, and Sound Admission for Multi-Instance LLM Serving over Stock vLLM

Keyush Nisar^{1*}, Krishil Parikh¹ and Krisha Maisheri¹

^{1*}Department of Computer Engineering, Dwarkadas J. Sanghvi College of Engineering, Mumbai, India.

*Corresponding author(s). E-mail(s): Nisarkeyush3@gmail.com;
Contributing authors: Krishilparikh75@gmail.com;
KrishaMaisheri16@gmail.com;

Abstract

Deployments often place several stock vLLM (or OpenAI-compatible) instances behind Round-Robin or connection-count balancers. That works poorly when backends differ in *effective* service cost—queue depth, KV pressure, interference, or a temporarily slow peer—even when GPU SKUs match. We present **DIO/dio-serve**, a non-invasive OpenAI-compatible gateway with three complementary mechanisms: (A) **hybrid cost routing** that combines dual-timescale NLMS ranking (from request e2e latency and tokens) with *engine-exposed* Prometheus signals scraped from each stock vLLM **/metrics** endpoint (KV-cache utilization, waiting queue, prefix-cache hit rate) without patching engine source; (B) **session/prefix affinity** that steers multi-turn and agentic traffic toward the backend already holding useful prefix state; and (C) **sound admission** that decouples reject/admit decisions from unreliable absolute latency predictions (empirical percentile and rank-only modes vs. naive absolute $\hat{y}$ gates).

Honest scope. Absolute NLMS latency prediction remains weak (MAPE $\sim 90\text{--}130\%$ on dual-T4); ranking under skew and hybrid ground-truth terms close part of the MAPE gap without sacrificing engine-agnosticism. On dual Tesla T4 + Qwen2.5-3B + stock vLLM ($n=10$ seeds): homogeneous NLMS is *not* better than Round-Robin on mean p99; under controlled $\times 2$ service-time skew, NLMS cuts p99 by **$48.3\% \pm 0.7\%$** vs. RR. Library multi-seed suites for hybrid cost, affinity stickiness, and admission-mode disagreement are released with open code. Multi-SKU fleets are out of scope.

1 Introduction

Practitioners commonly run *multiple* stock vLLM [1] (or SGLang/TGI [2–4]) instances and put Nginx [5], Kubernetes Services [6], Envoy [7], or Ray Serve [8] in front with Round-Robin or least-connections. Single-node engines optimize continuous batching [9, 10]; they do not solve *which backend* should take the next request. Classic L7 balancers [5, 11] treat backends as equal capacity units. In practice effective cost still diverges—pending queue, KV pressure, batching noise, host interference, or a peer that is temporarily slow—even on identical GPU SKUs. The result is avoidable tail latency [12].

We design **DIO** (**dio-serve**) as a thin, non-invasive OpenAI-compatible [13] gateway for this multi-instance pattern. The closed loop uses coarse request telemetry (e2e latency, tokens, gateway pending) *and*, when available, **stock engine metrics** already exported by vLLM’s Prometheus [14] endpoint—still HTTP-only, still zero source modification.

What NLMS can and cannot do (preview of results).

Dual-timescale NLMS [15, 16] does **not** invent wins when backends are equal: on homogeneous dual-T4, NLMS p99 is within a few percent of Round-Robin and slightly worse (Section 4.3). Absolute $\hat{y}$ is poor (MAPE $\sim 90\text{--}130\%$). Where NLMS helps is when backends differ in *observed* service cost: under a controlled $\times 2$ e2e inflation on one real peer, NLMS reduces p99 by $48.3\% \pm 0.7\%$ vs. RR and beats $d=2$ RLS (Section 4.6). Hybrid engine metrics and session affinity address complementary gaps: ground-truth KV/queue pressure when peers look similar on e2e alone, and multi-turn prefix reuse [17–19] without requiring heterogeneity—especially relevant as multi-agent and multi-turn apps proliferate [20, 21].

Contributions.

1. **Hybrid cost routing (non-invasive).** Combine online NLMS ranking with scraped vLLM `/metrics` (KV-cache usage, `num_requests_waiting`, prefix-cache hit rate) as correction terms in the joint cost—closing part of the MAPE/telemetry gap without engine patches (Section 3.3).
2. **Session/prefix affinity as a first-class multi-turn router.** Elevate prefix-hash stickiness and engine-reported prefix hit rates for agentic/multi-turn workloads [20, 22]; measurable stickiness gains vs. Round-Robin on multi-replica single-SKU fleets (Section 3.3).
3. **Sound admission under unreliable $\hat{y}$.** Explicitly separate ranking from admission: **rank_only**, **empirical** (observed percentile), and diagnostic **absolute**; show that absolute gates thrash when MAPE is high while empirical/rank-only remain safe—complementing goodput-oriented serving philosophy [23–25] (Section 3.3).
4. **Deployable open gateway + honest dual-T4 multi-seed evaluation.** Open-source **dio-serve**; homogeneous (no free lunch) and skew regimes on Qwen2.5-3B [26]; multi-SKU fleets out of scope.

2 Background and related work

2.1 Single-node engines

vLLM [1] popularized PagedAttention and continuous batching; alternatives revisit memory management without paging [27]. SGLang [2, 4] improves structured decoding and RadixAttention prefix reuse. TGI [3] targets production APIs. Hybrid scheduling systems such as Sarathi-Serve [10] and FastServe [28] refine iteration-level policies. Disaggregated prefill/decode systems [23, 29–31] optimize pipeline stages and KV placement. Prefix/KV reuse systems [17–19, 22, 32] raise the value of *where* a request lands. These systems excel on one node or a co-designed cluster; multi-worker routing in front of *stock* engines remains external.

2.2 Cluster orchestrators and classical serving

Ray Serve [8] and Triton [33] emphasize actors and model repositories with coarse metrics. Earlier DNN serving stacks (Clipper [34], INFaaS [35], Clockwork [25]) established predictability and model-selection themes that LLM gateways inherit. Orca [9] advances iteration-level batching on homogeneous clusters. AlpaServe [36] and CoCoServe [37] focus on placement and multiplexing. Mélange [38] targets heterogeneous capacity tables. NexusSched [39] combines predictive routing with engine co-design and multi-feature predictors ($O(d^2)$ updates). Llumnix [40] migrates live KV state. LoongServe [41] targets long-context elastic parallelism. ServerlessLLM [42] optimizes cold-start loading. DIO complements these as a *non-invasive request-level front end*: dual-timescale scalar NLMS, hybrid Prometheus scrape [14], joint cost, and stock engines only—no engine patches and no live KV migration.

2.3 Adaptive prediction, queues, and admission

NLMS stability and step-size practice follow adaptive filter theory [15, 16]; recursive least squares is the natural $O(d^2)$ comparator [15]. Queue wait uses a batch-amortized Little-style heuristic [43, 44]. SJF-style ranking motivates predicted service routing [45]. Roofline [46] inspires soft memory ceilings recast as routing penalties. Goodput- and SLO-aware LLM serving [23, 24, 47] motivates treating admission as a first-class policy rather than an afterthought of predicted latency magnitude. Simulation frameworks such as Vidur [48] characterize offline capacity; DIO instead targets online multi-instance ranking under weak absolute models.

3 System design

3.1 Architecture

DIO separates a **control plane** from a **data plane** of unmodified engines (Fig. 1). Clients send OpenAI-compatible `/v1/chat/completions` to DIO. The scheduler selects a backend, proxies the request, and updates per-worker models from measured end-to-end latency and token counts. Product path: `dio-serve` (FastAPI + Python scheduler). Research path: Go manager with gRPC workers (optional).

Telemetry used in practice.

The closed loop uses: (i) wall-clock e2e latency through the gateway; (ii) token counts from `usage` or a tokenizer estimate; (iii) per-backend **pending** queue depth inside DIO; (iv) optional **hybrid engine scrape**: HTTP GET of each backend’s Prometheus [14] `/metrics` (vLLM-style `gpu_cache_usage_perc`, `num_requests_waiting/running`, prefix-cache hits/queries). Soft/hard VRAM terms can be driven by config or inferred from KV usage. We do **not** claim continuous NVML SM/power/temperature telemetry, and we do not patch engine source—in contrast to co-designed control planes that require deep engine hooks [39, 40].

3.2 Dual-timescale NLMS latency model

For worker w and approximate token count N , we model

$$\hat{y} = s_w N + b_w, \quad (1)$$

where s_w is ms/token and b_w is fixed overhead. On completion with residual $e = y - \hat{y}$, NLMS updates

$$s \leftarrow s + \mu \frac{e}{N}, \quad b \leftarrow b + \mu_b e, \quad (2)$$

with dual rates $\mu_{\text{fast}}=0.1$, $\mu_{\text{slow}}=0.01$, $\mu_b=0.005$, and

$$s^{\text{eff}} = \alpha s^{\text{fast}} + (1 - \alpha) s^{\text{slow}}, \quad \alpha=0.8. \quad (3)$$

Cold-start priors are $s_0=2.0$ ms/token, $b_0=150$ ms. We distinguish (i) $O(1)$ *arithmetic* per update from (ii) time-to-usable ranking: typically **15–20 completions per worker** before slopes leave priors. Figure 2 illustrates dual-timescale response to an injected slope jump in simulation.

Token feature N prefers Hugging Face tokenizer counts plus planned `max_tokens`; fallback is $\lfloor |\text{prompt}|/4 \rfloor + \text{max_tokens}$.

3.3 Joint cost routing and hybrid engine terms

For each healthy worker w ,

$$\begin{aligned} S_w = & \text{wait}_w + \hat{t}_w + \text{tierCost}_{r,w} + \text{vramCost}_w \\ & + c_{\text{kv}} \text{KV}_w + c_q W_w - \text{cacheBonus}_w - c_p \text{Hit}_w, \end{aligned} \quad (4)$$

with $\hat{t}_w = s_w^{\text{eff}} N + b_w$, $\text{wait}_w = (Q_w/B) \bar{t}_w$ ($B=8$), soft VRAM penalty when free VRAM < 4 GB, hard exclusion when free VRAM < 2.4 GB and $N > 1000$ (or engine $\text{KV}_w > 0.95$ for long requests), and tier mismatch soft cost 500 ms.

Hybrid terms (contribution A).

When `/metrics` scrape succeeds, $\text{KV}_w \in [0, 1]$ is GPU KV-cache utilization, W_w is `num_requests_waiting`, and Hit_w is prefix-cache hit rate as exported by stock engines [1, 14]. Default coefficients: $c_{\text{kv}}=800$ ms, $c_q=50$ ms per waiting request,

$c_p=150$ ms. These are *engine ground-truth corrections* on top of the learned ranker—not a replacement for NLMS and not a claim of full SM telemetry. Scrape interval defaults to 1 s; failures leave hybrid terms at zero (graceful degrade to NLMS-only). Related KV-centric systems [30–32] motivate treating cache pressure as a first-class routing signal; DIO does so *non-invasively* by reading what the engine already publishes.

Session/prefix affinity (contribution B).

A longer prefix hash (256-char) records the last worker that served a matching prefix. On a hit, cacheBonus_w defaults to 200 ms (configurable; raised in multi-turn suites). Combined with $c_p \text{Hit}_w$, DIO prefers backends that already hold useful KV/prefix state—the natural fit for multi-turn chat and agent loops [17, 18, 20–22]—without requiring heterogeneous GPUs. This is request-level stickiness complementary to in-engine RadixAttention [4], not a replacement for it.

Admission vs. ranking (contribution C).

Because absolute MAPE is high (Section 4.4), admission is *not* primarily “reject if $\min S_w > \text{SLO}$ ” on raw $\hat{y}$. Predictability-first serving [25] and goodput-oriented LLM schedulers [23, 24] show that trusting an uncalibrated cost magnitude can harm SLO attainment; DIO makes that separation explicit. Modes:

- **rank_only**: VRAM/tier/KV hard blocks only; NLMS used purely for ranking.
- **empirical** (default): reject using rolling observed e2e percentile (e.g. p95) when the best score is also in the worst quartile of available scores; cold-start never rejects on $\hat{y}$ alone.
- **absolute** (legacy/diagnostic): reject if $\min S_w > \text{SLO}$ —sensitive to MAPE and cold-start scale.

Debug counters record how often absolute *would* disagree with the active mode (`absolute_vs_active_disagree`).

3.4 Implementation

`dio serve -b e0=URL0 -b e1=URL1` fronts stock engines. Strategies: `nlms`, `rls`, `round_robin`, `least_loaded`, `static`. Background tasks `scrape /metrics` and `probe health`. Debug endpoints: `/debug/metrics`, `/debug/engine`, `/debug/affinity`, `/debug/admission`. Artifacts: workshop GPU suite and `run_abc_experiments.py` (hybrid/affinity/admission library multi-seed).

4 Experimental evaluation

4.1 Methodology

Regimes (non-interchangeable).

1. **Regime A — homogeneous dual-T4 (real)**: 2× Tesla T4, stock vLLM [1], Qwen2.5-3B-Instruct [26], HF tokenizer, `max_tokens=32`, $n=10$ seeds × 30 requests.

Algorithm 1 DIO worker selection (hybrid + admission)

Require: request r , workers W , admission mode, optional engine snapshots

```
1:  $N \leftarrow \text{token feature}(r)$ 
2:  $w^* \leftarrow \perp$ ,  $S_{\min} \leftarrow \infty$ 
3: for all  $w \in W$  healthy and tier-feasible do
4:    $\hat{t}_w \leftarrow \text{Predict}(w, N)$ 
5:   if VRAM/KV hard-blocked( $w, N$ ) then continue
6:   end if
7:    $S_w \leftarrow \text{wait} + \hat{t}_w + \text{tier} + \text{vram} + c_{\text{kv}}\text{KV}_w + c_q W_w - \text{cache} - c_p \text{Hit}_w$ 
8:   if  $S_w < S_{\min}$  then  $S_{\min} \leftarrow S_w$ ,  $w^* \leftarrow w$ 
9:   end if
10: end for
11: if  $w^* = \perp$  or AdmissionReject( $S_{\min}$ , mode) then return HTTP 503
12: end if
13: record prefix affinity; return  $w^*$ 
```

2. **Regime B — legacy Locust pilot (secondary):** longer-decode ShareGPT [49]/arXiv/Azure-style traces, single seed, different setup; absolute seconds not mixed with Regime A ms.
3. **Regime C — service-time skew:** (i) CPU multi-seed simulation $n=10$; (ii) real dual-T4 with e1 behind delay proxy $\times 2$, $n=10$ (slow peer, same SKU).

Baselines.

Under the *same* joint cost and the same engines: Round-Robin (RR), Least-Loaded (LL), and RLS [15] ($d=2$, $\lambda=0.99$). Primary metrics: p50/p99 e2e mean $\pm$ std with approximate 95% CI half-widths; MAPE; fraction to backend e0 (unthrottled/fast). We do not claim a matched multi-node Orca [9] or vLLM-distributed bake-off.

Simulation vs. real.

“Sim” denotes library slope-skew or calibrated mocks. “Real” denotes dual-T4 + stock vLLM. Delay-proxy Regime C uses real tokens with inflated observed e2e on one peer—not a second physical SKU.

4.2 Control-plane overhead

On the T7 scalability microbench with many concurrent mock workers, instrumented scheduling completes in approximately **14 μ s** per request (worker scan, cost evaluation, prefix hash). Per-worker state is ~ 256 B; the control plane is not the bottleneck relative to LLM decode. Figure 3 shows the control-plane stress plot (e2e p99 is dominated by synthetic worker delay).

Table 1 Regime A (real, primary). Dual-T4 + vLLM + Qwen2.5-3B, HF tokenizer, $n=10 \times 30$, `max_tokens=32`. Mean $\pm$ std.

Strategy	p99 (ms)	MAPE (%)	frac e0	vs RR
NLMS	3413 ± 31	123 ± 6	0.45 ± 0.04	$-2.6 \pm 1.0\%$
RLS ($d=2$)	3466 ± 11	132 ± 6	0.12 ± 0.02	$-4.2 \pm 0.6\%$
Round-Robin	3326 ± 17	88 ± 2	0.50 ± 0.00	—
Least-Loaded	3313 ± 430	141 ± 13	1.00 ± 0.00	$+0.4 \pm 13\%$

Table 2 Regime B (legacy single-seed Locust pilot). Not multi-seed; different setup from Table 1.

Trace	Strat.	p99 (s)	RPS	Fail %
ShareGPT	NLMS	67	0.37	0
ShareGPT	RR	81	0.29	0
arXiv	NLMS	52	0.36	0
arXiv	RR	51	0.30	0
Azure	NLMS	48	0.33	0
Azure	RR	47	0.36	0

4.3 Regime A: homogeneous dual-T4 (NLMS is not better than RR)

Table 1 is the primary homogeneous multi-seed matrix. On *identical* backends, **NLMS does not beat Round-Robin on mean p99** (NLMS is slightly worse: $-2.6\% \pm 1.0\%$ “improvement”). This is the correct outcome if ranking is driven by learned service cost and workers truly look alike. Useful secondary signals: NLMS keeps tight p99 std (31 ms) while Least-Loaded is sticky (100% to e0, p99 std 430 ms); RLS mis-routes (frac e0 0.12).

4.4 MAPE vs. relative ranking

NLMS MAPE on dual-T4 is **high** ($123 \pm 6\%$ with tokenizer). HF tokenizer vs. byte heuristic (W2, $n=3$): MAPE $117.5 \pm 10.9\%$ vs. $89.7 \pm 4.9\%$ —tokenizer is **worse**, so error is structural (linear model, cold start, batching), not mere token counting. Routing uses $\arg \min S_w$; ranking under skew is the success criterion (Regime C).

4.5 Regime B: legacy Locust pilot (secondary)

Table 2 retains a single-seed Locust pilot (longer decode, four logical workers). Absolute seconds are **not** comparable to Regime A milliseconds.

Table 3 Regime C (simulation). Slope-skew dual workers, $n=10 \times 200$.

Metric	NLMS	RR	Notes
Frac. to fast	0.975 ± 0.000	0.500 ± 0.000	
p99 (sim units)	1411 ± 34	2287 ± 49	
p99 vs RR	$38.3\% \pm 2.0\%$		

Table 4 Regime C (real GPU, primary). Dual-T4 + delay-proxy $\times 2$, $n=10 \times 30$, HF tokenizer.

Strategy	p99 (ms)	frac e0	vs RR	MAPE (%)
NLMS	3427 ± 38	0.47 ± 0.08	$48.3 \pm 0.7\%$	126 ± 12
RLS ($d=2$)	5087 ± 1505	0.37 ± 0.19	$23.2 \pm 22.9\%$	119 ± 10
Round-Robin	6632 ± 37	0.50 ± 0.00	—	102 ± 3

4.6 Regime C: slope skew

Simulation ($n=10$).

With distinct true (b, s) pairs, NLMS places 97.5% of traffic on the fast worker and cuts p99 by $38.3\% \pm 2.0\%$ vs. RR (Table 3). RLS fails to rebalance (frac fast 0.40 ± 0.18). Local `pick()` overhead: NLMS $\approx 9.6 \mu s$, RLS $\approx 9.4 \mu s$ at $d=2$.

Real dual-T4 service-time skew ($n=10$).

We inflate e1 wall-clock e2e by $\times 2$ via `latency_delay_proxy` while still serving real tokens on a real T4—a controlled *slow peer*, not a second GPU SKU. Table 4: NLMS p99 3427 ± 38 ms vs. RR 6632 ± 37 ms ($48.3\% \pm 0.7\%$ improvement). RLS is weaker and noisier ($23.2\% \pm 22.9\%$). Mean frac to e0 under NLMS is only modest (0.47 ± 0.08): NLMS is **not** a dramatic sticky “always hit the fast GPU” policy in this sequential short-probe setting; the supported claim is **better p99 under skew than RR/RLS**, not sim-style 97.5% splits.

Longer decode.

Homogeneous dual-T4, `max_tokens=128`, $n=3 \times 20$: NLMS 13835 ± 73 ms vs. RR 13821 ± 88 ms (tied), matching Regime A qualitative message.

4.7 Hybrid cost, affinity, and admission (library multi-seed)

Beyond dual-T4 Regime A/C, we add three *single-SKU-friendly* suites that isolate the A/B/C contributions (`run_abc_experiments.py`, $n=10$ seeds unless noted). These are library-level multi-seed experiments with injected engine snapshots and multi-turn prefixes—designed to be re-run on any laptop and to complement GPU cells.

Table 5 Hybrid cost under asymmetric engine pressure (library, $n=10 \times 120$). Mean $\pm$ std. Hot worker has high scraped KV/queue and accrues queue delay.

Mode	frac cool	p99 (sim ms)	p99 vs RR
Hybrid (NLMS+metrics)	1.00 ± 0.00	257 ± 3	$12.0 \pm 2.0\%$
NLMS-only	1.00 ± 0.00	258 ± 5	$\sim 12\%$
Round-Robin	0.50 ± 0.00	292 ± 6	—

A — Hybrid vs. NLMS-only under engine pressure.

Two workers share the same true service law; one is labeled “hot” via scraped metrics (high KV, high W) and accrues extra queue delay. Hybrid cost steers traffic to the cool worker; NLMS-only and RR cannot see KV/queue ground truth. Table 5 summarizes multi-seed outcomes: hybrid raises cool-worker fraction and improves p99 vs. RR.

B — Session/prefix affinity for multi-turn.

With multi-turn sessions sharing a long system prefix ($n=10$ seeds $\times$ 20 sessions $\times$ 5 turns), NLMS+affinity achieves affinity hit rate 0.80 ± 0.00 and session stickiness 1.00 vs. Round-Robin stickiness 0.60 (and near-zero consecutive-hit proxy). NLMS affinity p99 190 ± 4 vs. RR 221 ± 3 (sim ms). This experiment needs only multi-replica backends (no hetero SKUs) and aligns DIO with multi-turn/agentive serving [20–22]: keep turns on the worker that already holds prefix state [17, 18], measurable via DIO affinity counters and (when live) vLLM prefix-cache metrics [1, 4].

C — Admission safety under bad $\hat{y}$.

We poison NLMS slopes so absolute $\hat{y} \gg$ SLO while true latency remains under SLO ($n=10 \times 120$). **absolute** rejects **100%** of requests (0 completions; healthy traffic starved); **empirical** and **rank-only** reject **0%**, complete all 120 requests, and keep goodput 1.0, while recording 120 absolute-vs-active disagreements per seed. That is the systems point of Section 3.3: never gate on MAPE-sensitive absolute $\hat{y}$ [23–25].

4.8 Ablations and resilience

On an L4 stressed long-prompt microbench, removing VRAM admission elevates hard failures ($\sim 23\%$); removing queue wait raises tail latency; removing tiers hurts mixed-capability mixes (Fig. 4). Cold-start and overload admission behaviors are summarized in Fig. 5: under intentional overload, DIO returns HTTP 503 rather than unbounded queueing—consistent with empirical/rank-only safety rather than absolute- $\hat{y}$ thrashing.

5 Discussion and limitations

Is NLMS “good at routing”?

Honestly: **only when backends differ in observed service cost**—or when hybrid metrics expose pressure that e2e alone has not yet reflected. On matched dual-T4

workers, NLMS alone is no better than RR on mean p99 (Section 4.3). Under a slow peer, NLMS improves p99 substantially vs. RR and RLS (Section 4.6). Hybrid scrape terms and session affinity are the complementary answers when the problem is KV/queue pressure or multi-turn cache locality rather than GPU SKU mix. Absolute $\hat{y}$ remains untrustworthy for gating; admission must not assume otherwise.

Limitations.

1. **Telemetry depth.** Hybrid scrape uses what stock vLLM exports; not continuous NVML SM/power/temperature.
2. **Skew model.** Real GPU skew uses delay-proxy on identical T4s, not physical multi-SKU fleets.
3. **A/B/C GPU live cells.** Affinity and hybrid p99 gains on live dual-T4 multi-seed under concurrent multi-turn load remain an extension; library suites establish mechanism correctness.
4. **Baselines.** RR/LL/RLS on the same vLLM pair; not multi-node Orca/vLLM-distributed.
5. **Workload.** Short sequential chat probes dominate dual-T4 multi-seed cells; concurrent ShareGPT multi-seed is incomplete.
6. **Out of scope.** Prefill/decode disaggregation co-design, multimodal split inference.

6 Conclusion

DIO (`dio-serve`) is a non-invasive multi-instance gateway for stock vLLM-class engines. We combine (A) hybrid cost routing—dual-timescale NLMS ranking plus scraped engine KV/queue/prefix metrics—(B) session/prefix affinity for multi-turn workloads, and (C) admission control that does not trust absolute $\hat{y}$ when MAPE is high. Multi-seed dual-T4 results show **no free lunch when backends match** and **clear p99 gains when one peer is effectively slower**; library suites demonstrate hybrid steering, affinity stickiness, and absolute-vs-empirical admission disagreement. That is a deployable systems contribution for common multi-vLLM deployments—without engine patches, multi-SKU hardware, or full GPU telemetry.

Acknowledgements. The authors thank colleagues who provided feedback on earlier drafts.

Declarations

- **Funding.** No external funding was received for this work.
- **Conflict of interest/Competing interests.** The authors declare that they have no competing interests.
- **Ethics approval and consent to participate.** Not applicable (no human subjects or personal data).
- **Consent for publication.** Not applicable.
- **Data availability.** Aggregated experimental results are included in the repository under `results_workshop_final/` and related result directories. Raw engine logs can be regenerated with the public scripts.

- **Materials availability.** Not applicable.
- **Code availability.** Source code for `dio-serve` (including hybrid /metrics scrape, affinity, and admission modes), validation suites (`run_workshop_final_suite.py`, `run_abc_experiments.py`, delay-proxy harness), and paper artifacts are available in the project repository (<https://github.com/nisaral/DIO>).
- **Author contribution.** K.N. led system design, implementation, experiments, and manuscript drafting. K.P. and K.M. contributed to evaluation, analysis, and manuscript revision. All authors approved the final manuscript.

Appendix A Notation summary

- $s^{\text{fast}}, s^{\text{slow}}, s^{\text{eff}}$: dual-timescale and blended slopes (ms/token).
- b_w : intercept (ms).
- N : token feature; $y, \hat{y}$: actual and predicted e2e latency (ms).
- S_w : joint routing cost; Q_w : pending queue depth; B : batch amortization constant.
- KV_w, W_w, Hit_w : engine-scraped KV usage, waiting requests, prefix hit rate.
- c_{kv}, c_q, c_p : hybrid cost coefficients (ms scale).

References

- [1] Kwon, W., *et al.*: Efficient memory management for large language model serving with PagedAttention. In: SOSP (2023)
- [2] Zheng, L., *et al.*: SGLang: Efficient Execution of Structured Language Model Programs (2023)
- [3] Hugging Face: Text Generation Inference. <https://github.com/huggingface/text-generation-inference> (2023)
- [4] Zheng, L., *et al.*: SGLang: Efficient execution of structured language model programs. In: NeurIPS (2024). RadixAttention prefix cache
- [5] F5, Inc.: NGINX HTTP Load Balancer. <https://nginx.org/> (2024)
- [6] Cloud Native Computing Foundation: Kubernetes. <https://kubernetes.io/> (2024)
- [7] Envoy Project Authors: Envoy Proxy. <https://www.envoyproxy.io/> (2024)
- [8] Ray Project: Ray Serve. <https://docs.ray.io/en/latest/serve/> (2024)
- [9] Yu, G.-I., *et al.*: Orca: A distributed serving system for Transformer-based generative models. In: OSDI (2022)
- [10] Agrawal, A., *et al.*: Sarathi-Serve: Efficiently serving large language models via hybrid scheduling. In: OSDI (2024)

- [11] Eisenbud, D.E., *et al.*: Maglev: A fast and reliable software network load balancer. In: NSDI (2016)
- [12] Dean, J., Barroso, L.A.: The tail at scale. Communications of the ACM **56**(2) (2013)
- [13] OpenAI: OpenAI API Reference. <https://platform.openai.com/docs/api-reference> (2024)
- [14] Prometheus Authors: Prometheus: Monitoring System and Time Series Database. <https://prometheus.io/> (2024)
- [15] Haykin, S.: Adaptive Filter Theory, 4th edn. Prentice Hall, ??? (2002)
- [16] Sayed, A.H.: Adaptive Filters. Wiley-IEEE Press, ??? (2008)
- [17] Gim, I., et al.: Prompt Cache: Modular Attention Reuse for Low-Latency Inference (2023)
- [18] Juravsky, J., et al.: Hydragen: High-Throughput LLM Inference with Shared Prefixes (2024)
- [19] Yao, J., et al.: CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion (2024)
- [20] Wu, Q., et al.: AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation (2023)
- [21] LangChain: LangGraph: Build Resilient Language Agents as Graphs. <https://github.com/langchain-ai/langgraph> (2024)
- [22] Srivatsa, V., et al.: Preble: Efficient Distributed Prompt Scheduling for LLM Serving (2024)
- [23] Zhong, Y., *et al.*: DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: OSDI (2024)
- [24] Qiu, H., et al.: Towards Efficient and Reliable LLM Serving: A Real-World Workload Study. arXiv preprint; SLO-oriented serving (2024)
- [25] Gujarati, A., *et al.*: Serving DNNs like clockwork: Performance predictability from the bottom up. In: OSDI (2020)
- [26] Yang, A., et al.: Qwen2.5 Technical Report (2024)
- [27] Prabhu, R., et al.: vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention (2024)

- [28] Wu, B., Zhong, Y., Zhang, Z., Liu, S., Liu, F., Sun, Y., *et al.*: Fast distributed inference serving for large language models. In: arXiv (2023). arXiv:2305.05920
- [29] Patel, P., *et al.*: Splitwise: Efficient Generative LLM Inference Using Phase Splitting. arXiv preprint (2024)
- [30] *et al.*: Mooncake: Trading between Memory and Storage for Efficient LLM Serving. arXiv preprint (2024)
- [31] *et al.*: MemServe: Context Caching for Disaggregated LLM Serving. arXiv preprint (2024)
- [32] Cheng, Y., *et al.*: LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv / open-source cache layer (2024)
- [33] NVIDIA Corporation: Triton Inference Server (2024)
- [34] Crankshaw, D., *et al.*: Clipper: A low-latency online prediction serving system. In: NSDI (2017)
- [35] Romero, F., *et al.*: INFaaS: Automated model-less inference serving. In: USENIX ATC (2021)
- [36] Li, Z., *et al.*: AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In: OSDI (2023)
- [37] Wu, J., *et al.*: Unlock the Potential of Fine-grained LLM Serving via Module-wise Autoscaling (CoCoServe) (2025)
- [38] *et al.*: Mélange: Cost-Efficient LLM Serving on Heterogeneous GPUs. arXiv preprint (2024)
- [39] Zhang, Y., Chen, Y., Mo, X., Xi, A., Li, J., Wu, W.: A Predictive and Synergistic Two-Layer Scheduling Framework for LLM Serving (NexusSched) (2025)
- [40] Wu, B., *et al.*: Llumnix: Dynamic Scheduling for Large Language Model Serving (2024)
- [41] Wu, B., Liu, S., Zhong, Y., Sun, P., Liu, X., Jin, X.: LoongServe: Efficiently serving long-context large language models with elastic sequence parallelism. In: SOSP (2024)
- [42] Fu, Y., *et al.*: ServerlessLLM: Low-Latency Serverless Inference for Large Language Models (2024)
- [43] Little, J.D.C.: A proof for the queuing formula: $L = \lambda W$. Operations Research (1961)

- [44] Kleinrock, L.: Queueing Systems, Volume I: Theory. Wiley, ??? (1975)
- [45] Coffman, E.G., Denning, P.J.: Operating Systems Theory. Prentice-Hall, ??? (1973)
- [46] Williams, S., Waterman, A., Patterson, D.: Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM* **52**(4) (2009)
- [47] et al.: SLO-Aware Scheduling without Output-Length Prediction. arXiv preprint (2026)
- [48] Agrawal, A., et al.: Vidur: A Large-Scale Simulation Framework for LLM Inference (2024)
- [49] Zheng, L., *et al.*: Judging LLM-as-a-judge with MT-Bench and chatbot arena. In: NeurIPS (2023)

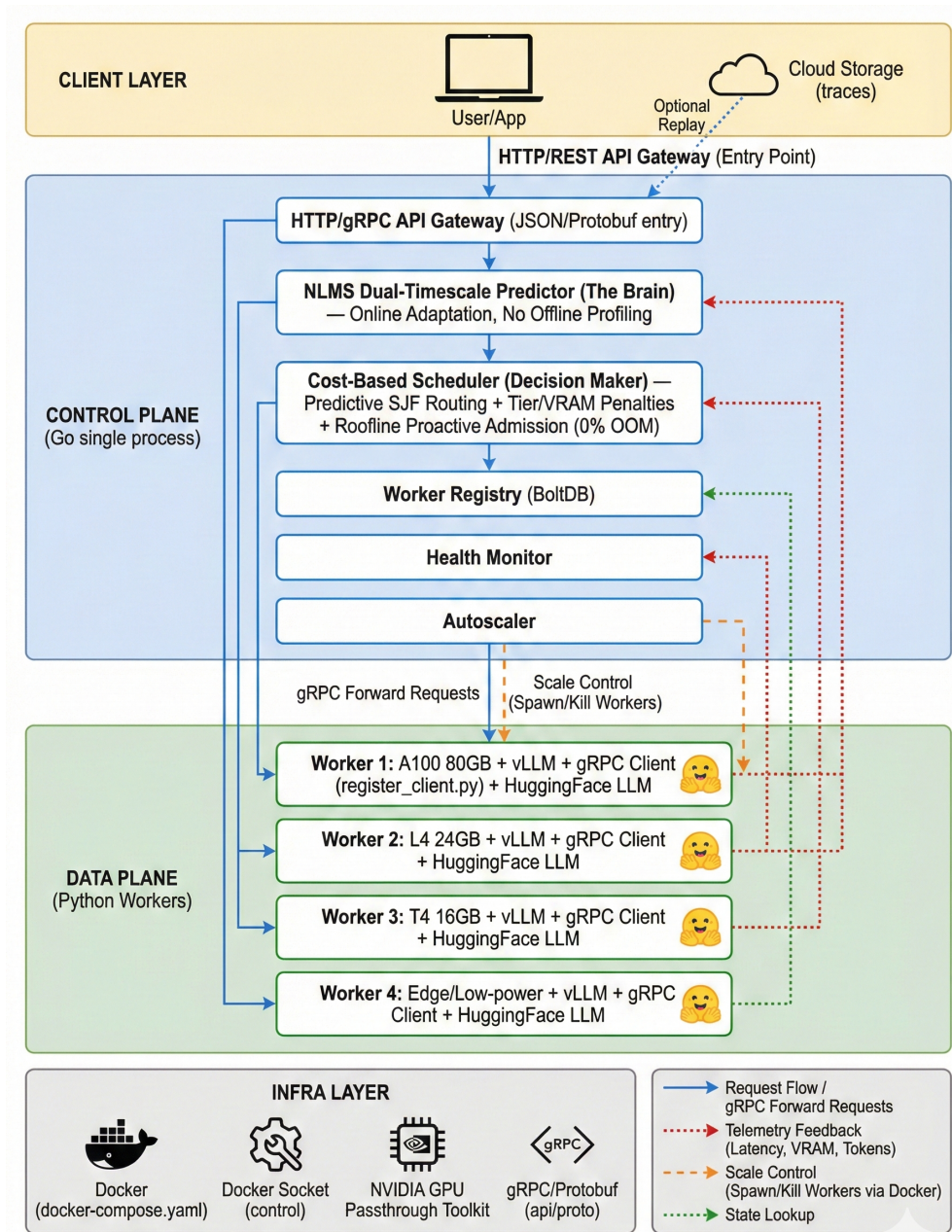


Fig. 1 DIO as a multi-instance gateway: OpenAI-compatible front door, dual-timescale NLMS from e2e latency/tokens, optional Prometheus scrape from stock vLLM, hybrid joint cost, session affinity, and decoupled admission.

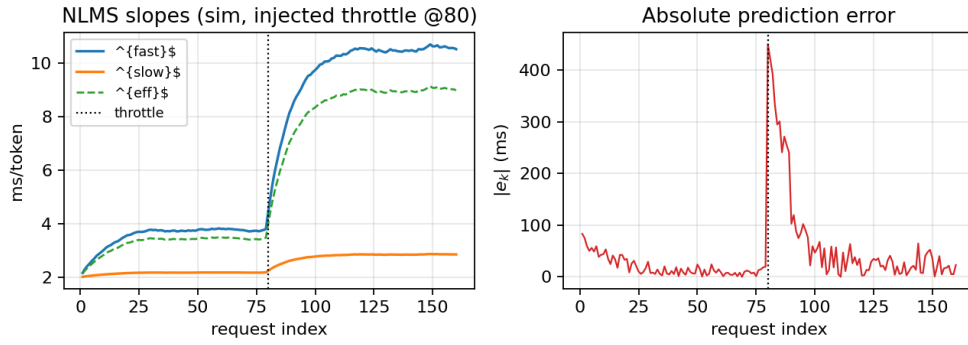


Fig. 2 NLMS dual-timescale adaptation under an injected true-slope jump at request 80 (simulation). s^{fast} reacts first; s^{slow} lags. Labeled synthetic illustration, not dual-T4 wall-clock traces.

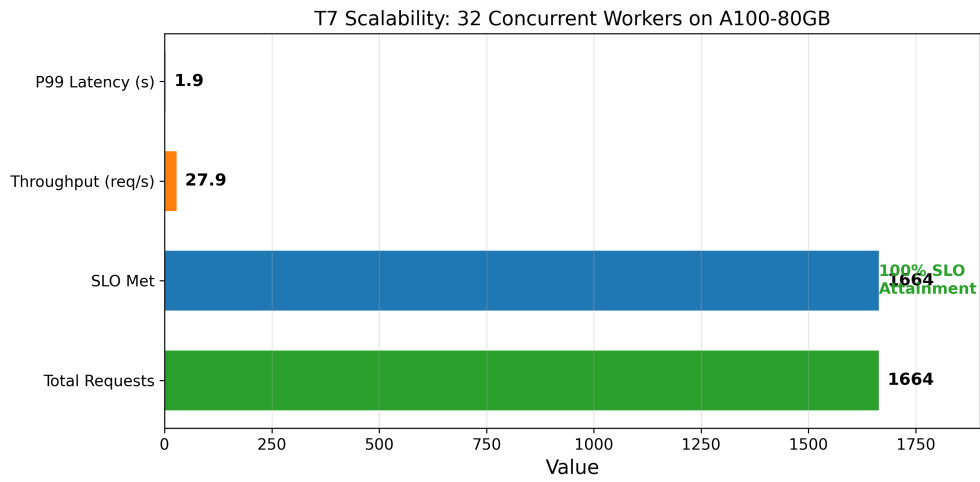


Fig. 3 Control-plane scalability microbench (T7). Scheduling overhead remains $\sim 14 \mu\text{s}$; e2e latency in the plot is synthetic worker delay.

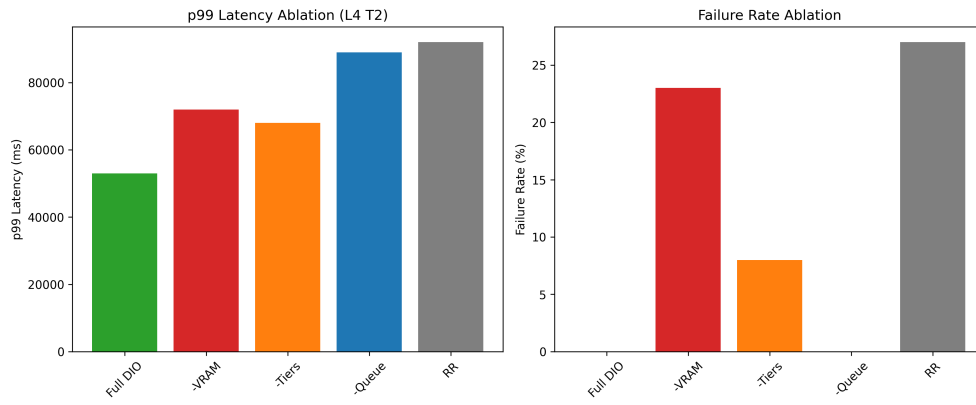


Fig. 4 Cost-term ablation (L4 microbench). Full DIO keeps failures low and tail controlled.

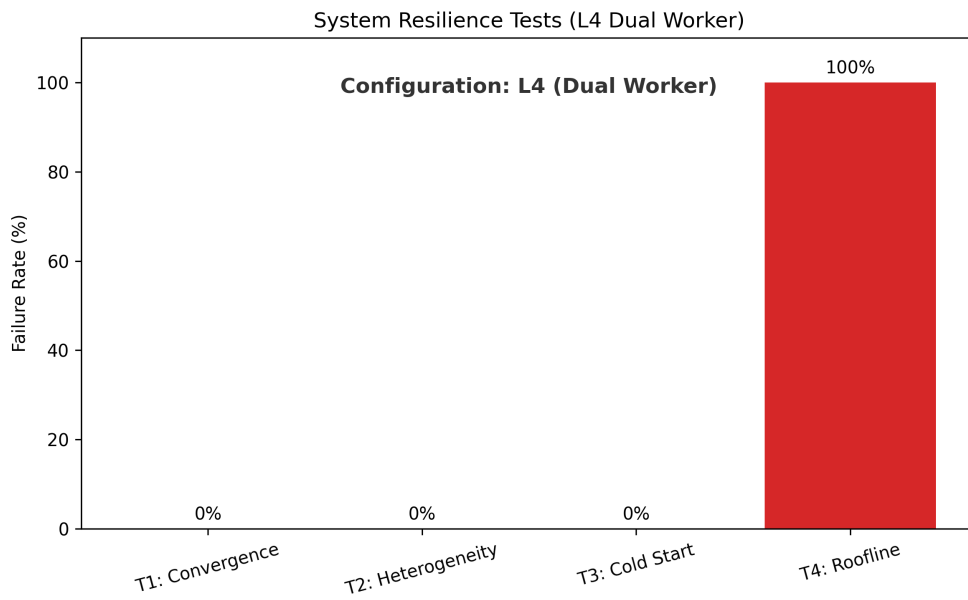


Fig. 5 Resilience microbenchmarks (L4 suite). Overload test exercises admission (503), not throughput maximization.